

Notas LABO 2 - Python

Introducción

Python es un lenguaje de programación de alto nivel que se destaca por su sintaxis clara y legible, lo que lo hace muy popular tanto para principiantes como para programadores experimentados. Los programas en Python suelen organizarse en archivos con extensión ".py" y se ejecutan utilizando un intérprete de Python.

Links útiles

<https://ellibrodepython.com/>

<https://numpy.org/>

<https://matplotlib.org/cheatsheets/>

https://pandas.pydata.org/docs/getting_started/index.html

Google Colab

Es lo que se va a utilizar para la clase. Se conecta a una maquina virtual en los servidores de Google, así que puedes correr código desde un teléfono si quisieras.

<https://colab.research.google.com/>

Permite delimitar pedazos de código para correrlos individualmente

Guarda archivos en .pynb y .py

Estructura de un código

Es recomendable que en cada línea haya una sola instrucción. Si una construcción es muy larga se puede dividir en varias líneas usando contra barra: \

No se usa punto y coma, sino el indentado

Elementos que lo componen

- [Palabras reservadas](#) (keywords)
- [Literales](#)

- [Operadores](#)
- [Delimitadores](#)
- [Funciones integradas](#) (built-in functions)

Palabras reservadas

Son las que forman el núcleo del lenguaje Python

```
False - await - else - import - pass - None - break - except - in - raise -  
True - class - finally - is - return - and - continue - for - lambda - try -  
as - def - from - nonlocal - while - assert - del - global - not - with -  
async - elif - if - or - yield
```

Estas palabras **no pueden utilizarse para nombrar otros elementos**, pero pueden aparecer en cadenas de texto.

Se utiliza lambda para funciones sin nombre

Literales

1. Literales numéricos: Representan valores numéricos, como enteros o números de punto flotante. Por ejemplo:
 - 42 (entero)
 - 3.14 (número de punto flotante)
 2. Literales de cadena de texto: Representan secuencias de caracteres. Pueden estar encerrados entre comillas simples (' '), comillas dobles (" "), o comillas triples (''' ' o '''). Por ejemplo:
 - 'Hola mundo'
 - "Python es genial"
 - '''Este es un literal de cadena multilinea'''
 3. Literales booleanos: Representan los valores verdadero (True) y falso (False). Por ejemplo:
 - True
 - False
 4. Literales de secuencia: Representan listas, tuplas y diccionarios. Por ejemplo:
 - [1, 2, 3] (lista)
 - (1, 2, 3) (tupla)
 - {'a': 1, 'b': 2, 'c': 3} (diccionario)
-

Operadores

Operadores aritméticos

Operador	Descripción	Uso
+	Suma entre los operandos	$12 + 3 = 15$
-	Resta entre los operandos	$12 - 3 = 9$
*	Multiplicación entre los operandos	$12 * 3 = 36$
/	División entre los operandos (para saber si un numero es par o impar)	$12 / 3 = 4$
%	Resto de la división entre los operandos	$16 \% 3 = 1$
**	Potencia de los operandos	$12 ** 3 = 1728$
//	División con resultado de número entero	$18 // 5 = 3$

Note

Para obtener el resultado en tipo flotante, uno de los operandos también debe ser de tipo flotante.

Operadores relacionales

Operador	Descripción	Uso
>	Devuelve <code>True</code> si el operando de la izquierda es mayor	$12 > 3 \rightarrow \text{True}$
<	Devuelve <code>True</code> si el operando de la derecha es mayor	$12 < 3 \rightarrow \text{False}$
==	Devuelve <code>True</code> si ambos operandos son iguales	$12 == 3 \rightarrow \text{False}$
>=	Devuelve <code>True</code> si el operando izq. es mayor o igual	$12 >= 3 \rightarrow \text{True}$
<=	Devuelve <code>True</code> si el operando der. es mayor o igual	$12 <= 3 \rightarrow \text{False}$
!=	Devuelve <code>True</code> si ambos operandos no son iguales	$12 != 3 \rightarrow \text{True}$

Operadores bit a bit, (para microcontroladores, no se va a usar)

Operador	Descripción	Uso
&	Operación AND bit a bit	$a \& b = 2$ ($10 \& 11 = 10$)
	Operación OR bit a bit	$a b = 3$ ($10 11 = 11$)

Operador	Descripción	Uso
$\wedge$	Operación XOR bit a bit	$a \wedge b = 1$ ($10 \wedge 11 = 01$)
$\sim$	Operación NOT bit a bit (invierte cada bit)	$\sim a = -3$ (Binario: $00000001 \rightarrow 11111100$)
$\gg$	Desplazamiento a la derecha (mueve bits a la derecha)	$a \gg b = 0$ ($00000010 \gg 00000011 = 0$)
$\ll$	Desplazamiento a la izquierda (mueve bits a la izquierda)	$a \ll b = 16$ ($00000001 \ll 00000100 = 00010000$)

Operadores de Asignación

Operador	Descripción
$=$	$a = 5 \rightarrow$ asigna el valor 5 a la variable a
$+=$	$a += 5$ equivale a $a = a + 5$
$-=$	$a -= 3$ equivale a $a = a - 3$
$*=$	$a = 3$ equivale a $a = a * 3$
$/=$	$a /= 3$ equivale a $a = a / 3$
$\% =$	$a \% = 3$ equivale a $a = a \% 3$
$**=$	$a = 3$ equivale a $a = a ** 3$
$//=$	$a //= 3$ equivale a $a = a // 3$
$\&=$	$a \&= 3$ equivale a $a = a \& 3$
$ =$	$a = 3$ equivale a $a = a 3$
$\wedge=$	$a \wedge= 3$ equivale a $a = a \wedge 3$
$\gg=$	$a \gg= 3$ equivale a $a = a \gg 3$
$\ll=$	$a \ll= 3$ equivale a $a = a \ll 3$

Operadores lógicos

Operador	Descripción	Uso
and	Devuelve True si ambos operandos son True	$a \text{ and } b$
or	Devuelve True si alguno de los operandos es True	$a \text{ or } b$
not	Devuelve True si alguno de los operandos es False	$\text{not } a$

Operadores de pertenencia

Un operador de pertenencia se emplea para identificar pertenencia en alguna secuencia (listas, strings, tuplas).

Operador	Descripción	Uso
in	Devuelve <code>True</code> si el valor especificado se encuentra en la secuencia. En caso contrario devuelve <code>False</code> .	a and b
not in	Devuelve <code>True</code> si el valor especificado no se encuentra en la secuencia. En caso contrario devuelve <code>False</code> .	not a

Delimitadores

Los delimitadores son los caracteres que permiten delimitar, separar o representar expresiones.

Símbolo	Nombre / Uso	Ejemplo
'	Comillas simples	'hola'
"	Comillas dobles	"hola"
#	Comentario	# esto es un comentario
\	Caracter de escape	"Hola\nMundo"
()	Paréntesis	print("ok")
[]	Lista o indexado	[1, 2, 3][0]
{}	Diccionario / set	{"a": 1}
:	Dos puntos	if x: ...
.	Punto / acceso a atributo	obj.attr
;	Separador de sentencias	x=1; y=2
@	Decorador	@classmethod
=	Asignación	x = 5
->	Anotación de retorno	def f() -> int:
+=	Suma acumulada	x += 1
-=	Resta acumulada	x -= 1
*=	Multiplicación acumulada	x *= 2
/=	División acumulada	x /= 3
//=	División entera acumulada	x //= 2
%=	Módulo acumulado	x %= 2
@=	Multiplicación de matrices acumulada	A @= B

Símbolo	Nombre / Uso	Ejemplo
&=	AND acumulado	x &= y
1=	OR acumulado	x 1= y
^=	XOR acumulado	x ^= y
>>=	Desplazamiento derecha acumulado	x >>= 1
<<=	Desplazamiento izquierda acumulado	x <<= 1
**=	Potencia acumulada	x **= 2

[1]

Funciones integradas

Una función es un bloque de instrucciones agrupadas, que permiten reutilizar partes de un programa.

```
abs() - dict() - help() - min() - setattr() - all() - dir() - hex() - next()
- slice() - any() - divmod() - id() - object() - sorted() - ascii() -
enumerate() - input() - oct() - staticmethod() - bin() - eval() - int() -
open() - str() - bool() - exec() - isinstance() - ord() - sum() -
bytearray() - filter() - issubclass() - pow() - super() - bytes() - float()
- iter() - print() - tuple() - callable() - format() - len() - property() -
type() - chr() - frozenset() - list() - range() - vars() - classmethod() -
getattr() - locals() - repr() - zip() - compile() - globals() - map() -
reversed() - import() - complex() - hasattr() - max() - round() - setattr() -
hash() - memoryview() - set()
```

Para sumar los elementos de un vector se puede usar sum, es mejor para la optimización

Important

Los nombres de las funciones integradas se pueden utilizar para nombrar variables, pero entonces las funciones ya no estarán disponibles en el programa.

Si se eliminan las variables, las funciones vuelven a estar disponibles.

Funciones

print()

La función `print()` admite varios argumentos seguidos.

En el programa, los argumentos deben separarse por comas.

Los argumentos se muestran en el mismo orden y en la misma línea, separados por espacios.

```
print("Hola", "Mundo")
```

```
# Output: Hola Mundo
```

Al final de cada `print()`, Python añade automáticamente un salto de línea

```
print("Hola")  
print("Mundo")
```

```
# Output:  
# Hola  
# Mundo
```

Para generar una línea en blanco, se puede escribir una orden `print()` sin argumentos

Se pueden utilizar variables:

```
nombre = "Maxi"  
print("¡Hola,", nombre, "!")
```

o, mejor cadenas:

```
nombre = "Juan"  
print(f"¡Hola, {nombre}!")
```

input()

La función `input()` permite obtener texto escrito por teclado:

```
print("¿Cómo se llama?")  
nombre = input()  
print(f"Un gusto, {nombre}")
```

Podemos aprovechar que a la función `input()` se le puede enviar un argumento que se escribe en la pantalla:

```
nombre = input("¿Cómo se llama? ")
print(f"Me alegro de conocerle, {nombre}")
```

Note

Al igual que `print()`, `input()` pone un salto de línea también

Important

Todo lo que se toma, por defecto, es string.
Se puede convertir a lo que necesites

Conversión de tipos

Antes de la función `input` le especificas el tipo:

```
cantidad = int(input(" Cuántos pesos tiene?: "))
print(f"{cantidad} pesos son {round(cantidad / 1000.0, 2)} dólares")
```

Variable `isinstance()`

Se utiliza para verificar el tipo específico de una variable

```
variable = 10
print(isinstance(variable, int)) # Esto imprimirá True si la variable es de
tipo entero (int), de lo contrario, imprimirá False.
```

`strip()`

Elimina los saltos de línea

Variables

Las variables en Python son como contenedores donde puedes almacenar datos

```
nombre = "Juan"
edad = 25
```

```
altura = 1.75
fecha_de_nacimiento="25/05/2000"
```

Una cosa interesante sobre Python es que no necesitas especificar el tipo de datos que contendrá la variable.

Listas

Las listas son estructuras de datos muy flexibles.

```
numeros = [1, 2, 3, 4, 5]
```

Se puede declarar una vacía y agregar elementos

```
frutas = []
frutas.append("manzana")
frutas.append("banana")
frutas.append("cereza")
```

Se puede acceder a elementos individuales de la lista utilizando su índice, comienza desde 0

```
frutas = ["manzana", "banana", "cereza"]
print(frutas[0]) # Imprime: manzana
print(frutas[1]) # Imprime: banana
```

También se puede acceder a elementos desde el final de la lista utilizando índices negativos:

```
frutas = ["manzana", "banana", "cereza"]
print(frutas[-1]) # Imprime: cereza
```

Slicing (rebanado) de listas

Se puede obtener una porción (slice) de la lista utilizando la sintaxis [inicio:fin]

```
numeros = [1, 2, 3, 4, 5]
print(numeros[1:3]) # Imprime: [2, 3]
```

Para tener en cuenta

El inicio se incluye, pero el fin no

Modificación de elementos

Se puede cambiar el valor de un elemento de la lista asignándole un nuevo valor

```
frutas = ["manzana", "banana", "cereza"]
frutas[1] = "kiwi"
print(frutas) # Imprime: ["manzana", "kiwi", "cereza"]
```

Borrado

Pueden eliminar elementos de la lista utilizando la palabra clave `del` o el método `remove()`.
Por ejemplo:

```
frutas = ["manzana", "banana", "cereza"]
del frutas[0] # Elimina la primera fruta
print(frutas) # Imprime: ["banana", "cereza"]

frutas.remove("banana") # Elimina la fruta "banana"
print(frutas) # Imprime: ["cereza"]
```

Note

La primera que se encuentre la borra

Si lo pones en while mientras exista, borra todo

Ojo

Cuando empiezas a borrar se empieza a mover la lista

Operaciones comunes con listas

`append()`

Agrega un ítem a la lista

`extend()`  `+`

Extiende una lista con los elementos de otra lista

len()

Para encontrar la longitud de la lista

```
print(len()) # Imprime cantidad de items
```

in()

Para buscar un elemento en la lista

Ejemplos:

```
frutas = ["manzana", "banana"]
frutas.append("cereza")
print(frutas) # Imprime: ["manzana", "banana", "cereza"]

otras_frutas = ["naranja", "uva"]
frutas.extend(otras_frutas)
print(frutas) # Imprime: ["manzana", "banana", "cereza", "naranja",
"uva"]

print(len(frutas)) # Imprime: 5

print("manzana" in frutas) # Imprime: True
```

Ordenamiento

sort()

Ordena de mayor a menor

```
numeros = [3, 1, 4, 1, 5, 9, 2]
numeros.sort()
print(numeros) # Imprime: [1, 1, 2, 3, 4, 5, 9]
```

sort(reverse=True)

Ordena de menor a mayor

```
numeros = [3, 1, 4, 1, 5, 9, 2]
numeros.sort(reverse=True)
```

```
print(numeros) # Imprime: [9, 5, 4, 3, 2, 1, 1]
```

Índice de un elemento

```
index(“”)
```

Para saber la posición de un ítem en la lista

```
frutas = ["manzana", "banana", "cereza"]
indice = frutas.index("banana")
print(indice) # Imprime: 1
```

Contar ocurrencias de un elemento

```
count()
```

```
numeros = [1, 2, 3, 4, 1, 2, 3, 1]
conteo = numeros.count(1)
print(conteo) # Imprime: 3
```

Reversión de una lista

```
reverse()
```

Da vuelta la lista

```
frutas = ["manzana", "banana", "cereza"]
frutas.reverse()
print(frutas) # Imprime: ["cereza", "banana", "manzana"]
```

Eliminación de todos los elementos

```
clear
```

```
frutas = ["manzana", "banana", "cereza"]
frutas.clear()
print(frutas) # Imprime: []
```

La lista queda vacía

Condicionales

Permiten controlar el flujo de ejecución de tu programa. Son bifurcaciones en el desarrollo del programa:

```
edad = 20

if edad >= 18:
    print("Eres mayor de edad.")
```

Si queremos ejecutar un bloque de código cuando la condición no es verdadera, se utiliza el `else`:

```
if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")
```

A veces, también queremos verificar múltiples condiciones. Para eso, usamos el `elif` (abreviatura de "else if")

```
if edad < 18:
    print("Eres menor de edad.")
elif edad == 18:
    print("Tienes exactamente 18 años.")
else:
    print("Eres mayor de edad.")
```

Finalmente, Python también nos permite combinar condiciones utilizando los operadores lógicos `and`, `or` y `not`. Por ejemplo:

```
temperatura = 25
llueve = True

if temperatura > 20 and not llueve:
    print("Es un buen día para salir.")
elif temperatura <= 20 or llueve:
    print("Mejor quedarse en casa.")
```

Si se quiere evaluar una condición, pero no se quiere hacer nada se usa `pass`, respetando el indentado

Cuestiones a tener en cuenta:

Lo que NO se permite es que en un mismo bloque haya instrucciones con distintos indentados. Dependiendo del orden de los indentado, el mensaje de error al intentar ejecutar el programa será diferente

Ciclos

for:

Es una estructura de control que nos permite iterar sobre una secuencia (como una lista, una tupla, un diccionario, un conjunto o una cadena de caracteres)

```
for variable in secuencia:
    # cuerpo del ciclo
```

- Se usa cuando sabes la cantidad de iteraciones

- Para recorrer iterables, listas, tuplas, dict, conjuntos, cadena de caracteres (string). UNA variable, sin auxiliares (sin i, i++)

- Se puede utilizar una variable auxiliar (ej. numero)

Ejemplos

Supongamos que tenemos una lista de números y queremos imprimir cada número:

```
numeros = [1, 2, 3, 4, 5]

for numero in numeros:
    print(numero)
```

También podemos usar `for` para iterar sobre una cadena de caracteres

```
cadena = "Python"

for caracter in cadena:
    print(caracter)
```

- El ciclo for en Python utilizarse con cualquier objeto iterable.

Si queremos calcular la suma de todos los números en una lista, usaremos un acumulador para almacenar la suma a medida que iteramos sobre la lista.

```
numeros = [1, 2, 3, 4, 5]
suma = 0 # Inicializamos el acumulador en 0

for numero in numeros:
    suma += numero # Añadimos cada número al acumulador

print("La suma de los números es:", suma)
```

Podemos usar un acumulador para contar cuántos elementos en una lista cumplen con una condición específica.

```
numeros = [1, 2, 3, 4, 5]
contador = 0 # Inicializamos el contador en 0

for numero in numeros:
    if numero > 3:
        contador += 1 # Incrementamos el contador cuando la condición se cumple

print("Hay", contador, "números mayores que 3.")
```

Supongamos que queremos concatenar todas las palabras en una lista en una sola cadena, separadas por espacios.

```
palabras = ["Hola", "mundo", "desde", "Python"]
cadena_concatenada = "" # Inicializamos la cadena vacía

for palabra in palabras:
    cadena_concatenada += palabra + " " # Añadimos cada palabra seguida de un espacio

print("La cadena concatenada es:", cadena_concatenada)
```

Otro ejemplo de acumulador es calcular el producto de todos los números en una lista.

```
numeros = [1, 2, 3, 4, 5]
producto = 1 # Inicializamos el acumulador en 1

for numero in numeros:
```

```
producto *= numero # Multiplicamos cada número al acumulador

print("El producto de los números es:", producto)
```

Imaginemos que queremos encontrar el número más grande en una lista de números. Usaremos una *variable testigo* para almacenar el mayor número encontrado hasta el momento.

```
numeros = [3, 7, 2, 8, 4, 10, 1]
maximo = numeros[0] # Inicializamos la variable testigo con el primer
elemento de la lista

for numero in numeros:
    if numero > maximo:
        maximo = numero # Actualizamos la variable testigo si encontramos un
número mayor

print("El número más grande es:", maximo)
```

Supongamos que queremos verificar si hay algún número par en una lista de números. Usaremos una bandera para indicar si encontramos un número par.

```
numeros = [3, 7, 2, 8, 4, 10, 1]
hay_par = False # Inicializamos la bandera en False

for numero in numeros:
    if numero % 2 == 0:
        hay_par = True # Cambiamos la bandera a True si encontramos un
número par
        break # Podemos salir del ciclo una vez que encontremos un número
par
if hay_par:
    print("Hay al menos un número par en la lista.")
else:
    print("No hay números pares en la lista.")
```

Note

Variable booleana, tiene que pasar un flag (bandera)

Imaginemos que tenemos una matriz (lista de listas) y queremos sumar todos los elementos de la matriz.

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

suma_total = 0 # Inicializamos el acumulador en 0

for fila in matriz: # Primer ciclo recorre cada fila de la matriz
    for elemento in fila: # Segundo ciclo recorre cada elemento en la fila
        suma_total += elemento # Sumamos el elemento al acumulador

print("La suma de todos los elementos en la matriz es:", suma_total)
```

while:

Es una estructura de control que permite ejecutar un bloque de código repetidamente mientras una condición específica sea verdadera.

```
while condición:
    # cuerpo del ciclo
```

Important

Que tenga una condición de cierre, a prueba de errores.

Ejemplos

En este ejemplo, el ciclo while seguirá ejecutándose mientras contador sea menor que 5. En cada iteración, se imprime el valor de contador y luego se incrementa en 1. Cuando contador llega a 5, la condición `contador < 5` se vuelve falsa y el ciclo termina.

```
contador = 0

while contador < 5:
    print("Contador:", contador)
    contador += 1
```

En este caso, el ciclo while continuará hasta que el usuario escriba "salir". La función `input` solicita al usuario que ingrese un texto, y el ciclo se repite hasta que la entrada del usuario, convertida a minúsculas, sea igual a "salir".

```
# Ejemplo de un ciclo while con una condición de salida específica
respuesta = ""

while respuesta.lower() != "salir":
    respuesta = input("Escribe 'salir' para terminar el programa: ")
    print("Ingresaste:", respuesta)
```

Casos especiales

break

Termina el ciclo inmediatamente, independientemente de la condición.

```
contador = 0

while True:
    print("Contador:", contador)
    contador += 1

    if contador >= 5:
        break # Sale del ciclo cuando contador es mayor o igual a 5
```

continue

Salta el resto del bloque de código en la iteración actual y pasa a la siguiente iteración del ciclo.

```
contador = 0

while contador < 5:
    contador += 1

    if contador == 3:
        continue # Salta el resto del bloque cuando contador es 3

    print("Contador:", contador)
```

else

Python permite usar una cláusula else con un ciclo while. El bloque else se ejecuta cuando la condición del while se vuelve falsa, pero no si el ciclo se interrumpe con un break.

```
contador = 0

while contador < 5:
    print("Contador:", contador)
    contador += 1
else:
    print("El ciclo ha terminado")
```

Rangos

La función `range` en Python se utiliza para generar una secuencia de números. Es comúnmente utilizada en ciclos `for` para iterar un número específico de veces.

Estructura

```
range(inicio, fin, paso)
```

Donde:

- inicio es el número inicial de la secuencia (inclusive)
- fin es el número en el que la secuencia se detiene (exclusivo)
- paso es la diferencia entre cada par de números consecutivos en la secuencia

Important

`range` solo admite pasos de números enteros!

Cuando se pasa un solo argumento, `range` genera números desde 0 hasta fin -1, con paso 1.

```
for i in range(5):
    print(i) # La secuencia generada es [0, 1, 2, 3, 4]
```

Cuando se pasan dos argumentos, `range` genera números desde inicio hasta fin- 1, con paso 1.

```
for i in range(2, 5):
    print(i) # La secuencia generada es [2, 3, 4]
```

Cuando se pasan tres argumentos, `range` genera números desde inicio hasta fin- 1, incrementando por paso especificado.

```
for i in range(1, 10, 2):  
    print(i) # La secuencia generada es [1, 3, 5, 7, 9]
```

También puede trabajar con valores negativos para inicio, fin y paso.

```
for i in range(10, 0, -2):  
    print(i) # La secuencia generada es [10, 8, 6, 4, 2]
```

Pueden convertir un objeto `range` en una lista usando la función `list` :

```
lista = list(range(5))  
print(lista) # Obtenemos [0, 1, 2, 3, 4]
```

E iterar sobre índices de una lista:

```
mi_lista = ['a', 'b', 'c', 'd']  
  
for i in range(len(mi_lista)):  
    print(f"Índice {i} tiene el valor {mi_lista[i]}")
```

Objetos

Estructura de un objeto

Tipo	list, int, str...
Valor	[], 25, "hola"...
Identificador	125518484848

Objetos Mutables

Los objetos mutables son aquellos que pueden ser modificados después de ser creados. Podés cambiar su contenido sin necesidad de crear un nuevo objeto.

- Listas (list)
- Diccionarios (dict)

- Conjuntos (set)

Ejemplos de mutabilidad

Si modificamos una lista y imprimimos su identificador

```
my_list = [1, 2, 3]

print(my_list) # Imprime: [1, 2, 3]
print(id(my_list)) # Imprime una dirección de memoria

my_list[0] = 10

print(my_list) # Imprime: [10, 2, 3]
print(id(my_list)) # Imprime la misma dirección de memoria
```

En cambio en una cadena de caracteres

```
my_str = "hello"

print(my_str) # Imprime: hello
print(id(my_str)) # Imprime una dirección de memoria

my_str = "world"

print(my_str) # Imprime: world
print(id(my_str)) # Imprime una dirección de memoria diferente
```

Problema del objeto mutable

Acá `lista_original` y `lista_nueva` apuntan al mismo objeto en memoria. Por lo tanto, cuando modificamos `lista_nueva`, también estamos modificando `lista_original`.

```
lista_original = [1, 2, 3]
lista_nueva = lista_original # Asignamos lista_original a lista_nueva

# Modificamos lista_nueva
lista_nueva.append(4)

# Observamos ambas listas
print("lista_original:", lista_original) # Output: [1, 2, 3, 4]
print("lista_nueva:", lista_nueva) # Output: [1, 2, 3, 4]
```

Solución del problema

Para solucionar el problema de la mutabilidad deberíamos realizar una copia con el método `copy()` :

```
lista_original = [1, 2, 3]
lista_nueva = lista_original.copy() # Creamos una copia de la lista original

# Modificamos lista_nueva
lista_nueva.append(4)

# Observamos ambas listas
print("lista_original:", lista_original) # Output: [1, 2, 3]
print("lista_nueva:", lista_nueva) # Output: [1, 2, 3, 4]
```

Objetos Inmutables

Los objetos inmutables son aquellos que, una vez creados, no pueden ser modificados. Si intentas cambiar el valor de un objeto inmutable, en realidad estarás creando un nuevo objeto.

- Números (enteros, flotantes)
- Cadenas de texto (str)
- Tuplas (tuple)

Importancia de la Inmutabilidad

- Seguridad y Consistencia: Los objetos inmutables no pueden ser modificados accidentalmente después de su creación.
- Hashing: Solo los objetos inmutables pueden ser usados como claves en un diccionario o elementos de un conjunto.
- Facilita el Depurado: El comportamiento de los objetos inmutables es más predecible y fácil de seguir en el código.

Conjuntos

Un conjunto es una colección de elementos únicos y no ordenados. A diferencia de las listas y las tuplas, **los conjuntos no permiten elementos duplicados**.

```
conjunto_vacio = set()
print(conjunto_vacio) # Output: set()
```

Podemos poner el dato que se te cante, muy útil

Podemos crear conjuntos a partir de otras colecciones como listas o cadenas:

```
lista = [1, 2, 3, 4, 4, 5]
conjunto_desde_lista = set(lista)
print(conjunto_desde_lista) # Output: {1, 2, 3, 4, 5}

cadena = "hola"
conjunto_desde_cadena = set(cadena)
print(conjunto_desde_cadena) # Output: {'h', 'o', 'l', 'a'}
```

Podemos añadir elementos a un conjunto utilizando el método `add()` y eliminarlos usando el método `remove()`

```
conjunto = {1, 2, 3}
conjunto.add(4)
print(conjunto) # Output: {1, 2, 3, 4}

conjunto.remove(2)
print(conjunto) # Output: {1, 3, 4}
```

Podemos verificar si un elemento está en un conjunto usando el operador `in`

```
conjunto = {1, 2, 3}
print(2 in conjunto) # Output: True
print(5 in conjunto) # Output: False
```

Operaciones de conjuntos

Unión

La unión de dos conjuntos contiene todos los elementos de ambos conjuntos.

Utilizamos el operador `|` o el método `union()` :

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}

union = conjunto1 | conjunto2
print(union) # Output: {1, 2, 3, 4, 5}
```

```
union = conjunto1.union(conjunto2)
print(union) # Output: {1, 2, 3, 4, 5}
```

Intersección

La intersección de dos conjuntos contiene solo los elementos que están en ambos conjuntos.

Utilizamos el operador `&` o el método `intersection()` :

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}

interseccion = conjunto1 & conjunto2
print(interseccion) # Output: {3}

interseccion = conjunto1.intersection(conjunto2)
print(interseccion) # Output: {3}
```

Diferencia

La diferencia entre dos conjuntos contiene los elementos que están en el primer conjunto pero no en el segundo.

Utilizamos el operador `-` o el método `difference()` :

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}

diferencia = conjunto1 - conjunto2
print(diferencia) # Output: {1, 2}

diferencia = conjunto1.difference(conjunto2)
print(diferencia) # Output: {1, 2}
```

Diferencia simétrica

La diferencia simétrica contiene los elementos que están en cualquiera de los conjuntos, pero no en ambos.

Utilizamos el operador `^` o el método `symmetric_difference()` :

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}
```

```
diferencia_simetrica = conjunto1 ^ conjunto2
print(diferencia_simetrica) # Output: {1, 2, 4, 5}
```

```
diferencia_simetrica = conjunto1.symmetric_difference(conjunto2)
print(diferencia_simetrica) # Output: {1, 2, 4, 5}
```

Subconjunto o superconjunto

Podemos verificar si un conjunto es subconjunto de otro usando `issubset()` y si es un superconjunto usando `issuperset()` :

```
a = {1, 2, 3}
b = {1, 2, 3, 4, 5}

print(a.issubset(b)) # Output: True
print(b.issuperset(a)) # Output: True
```

Diccionarios

Un diccionario es una colección desordenada, modificable e indexada de elementos. En lugar de usar índices numéricos como las listas, los diccionarios usan claves únicas para acceder a sus valores.

```
mi_diccionario = {
    "clave1": "valor1",
    "clave2": "valor2",
    "clave3": "valor3"
}
```

Diccionario con Elementos

Pueden crear un diccionario vacío y luego añadirle elementos:

```
diccionario_vacio = {}
```

O puedes crear un diccionario con elementos directamente:

```
mi_diccionario = {
    "nombre": "Juan",
    "edad": 25,
```

```
"ciudad": "Madrid"
}
```

Acceder a los Elementos del Diccionario

Para acceder a un valor en el diccionario, usas la clave correspondiente entre corchetes [].

```
print(mi_diccionario["nombre"]) # Output: Juan
print(mi_diccionario["edad"]) # Output: 25
```

Modificar Elementos del Diccionario

```
mi_diccionario["edad"] = 26
print(mi_diccionario["edad"]) # Output: 26
```

Añadir Nuevos Elementos al Diccionario

Para añadir un nuevo par clave-valor, simplemente asignas un valor a una nueva clave.

```
mi_diccionario["profesión"] = "Ingeniero"
print(mi_diccionario)
```

Eliminar Elementos del Diccionario

Puedes eliminar elementos del diccionario usando el método `pop()` o la palabra clave `del`.

```
mi_diccionario.pop("ciudad")
print(mi_diccionario)

del mi_diccionario["profesión"]
print(mi_diccionario)
```

Métodos Útiles de Diccionarios

- `keys()` : Devuelve una vista de todas las claves en el diccionario.
- `values()` : Devuelve una vista de todos los valores en el diccionario.
- `items()` : Devuelve una vista de todos los pares clave-valor en el diccionario.

```
print(mi_diccionario.keys()) # Output: dict_keys(['nombre', 'edad'])
print(mi_diccionario.values()) # Output: dict_values(['Juan', 26])
```

```
print(mi_diccionario.items()) } # Output: dict_items([('nombre', 'Juan'), ('edad', 6)])
```

Iterar Sobre un Diccionario

Podes iterar sobre un diccionario usando un bucle `for` . Puedes iterar sobre las claves, los valores o ambos.

```
for clave in mi_diccionario:  
    print(clave, mi_diccionario[clave])  
  
for clave, valor in mi_diccionario.items():  
    print(clave, valor)
```

- utilización es como un .JSON

- no podes poner una lista en una llave, tienen que ser objetos inmutables

- tenes que definir a que key, a que clave

Tuplas

Una tupla es una colección ordenada de elementos que puede contener diferentes tipos de datos (números, cadenas, listas, etc.).

La principal característica de las tuplas es que son inmutables, es decir, una vez creadas, no se pueden modificar (añadir, eliminar o cambiar sus elementos). Por eso mismo es que no se puede hacer ningún tipo de ordenación.

- La podes ordenar únicamente si la guardas en otra variable

Sintaxis Básica

Las tuplas se definen con paréntesis `()` y los elementos se separan por comas.

```
mi_tupla = (1, 2, 3)
```

Creación de una Tupla

Tupla Vacía

```
tupla_vacia = ()
```

Tupla con Elementos

```
mi_tupla = ("manzana", "banana", "cereza")
```

Para una tupla de un solo elemento, debes añadir una coma después del elemento

```
tupla_un_elemento = ("manzana",)
```

Acceder a los Elementos de una Tupla

```
print(mi_tupla[0]) # Output: manzana  
print(mi_tupla[1]) # Output: banana
```

Desempaquetado de Tuplas

Pueden asignar los elementos de una tupla a variables individuales.

```
fruta1, fruta2, fruta3 = mi_tupla  
print(fruta1) # Output: manzana  
print(fruta2) # Output: banana  
print(fruta3) # Output: cereza
```

Métodos y Operaciones con Tuplas

Las tuplas tienen algunos métodos integrados y soportan varias operaciones:

- `count(x)` : Devuelve el número de veces que x aparece en la tupla.
- `index(x)` : Devuelve el índice de la primera aparición de x en la tupla.

```
mi_tupla = (1, 2, 3, 4)  
print(mi_tupla.count(2)) # Output: 2  
print(mi_tupla.index(3)) # Output: 3
```

Inmutabilidad de las Tuplas

Una vez creada una tupla, no se pueden modificar sus elementos. Intentar hacerlo resultará en un error.

```
mi_tupla = (1, 2, 3)
mi_tupla[1] = 4 # Esto dará un error TypeError: 'tuple' object does not
support item assignment
```

Hacer una tupla vacía es bastante inútil

Usos Comunes de las Tuplas

Las tuplas se usan comúnmente cuando quieren almacenar una colección de elementos que no deben cambiar a lo largo del programa.

Por ejemplo, puedes usar tuplas para:

- Almacenar coordenadas (x, y).
- Almacenar días de la semana.
- Como claves en diccionarios (ya que las claves deben ser inmutables).

Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. Puedes definir una función una vez y luego llamarla tantas veces como sea necesario, lo que ayuda a mantener tu código limpio y organizado.

Podes llamarla cuantas veces quieras

Definición de una Función

```
def nombre_de_la_funcion():
    # Cuerpo de la funcion
    print("Hola, soy una funcion")
```

Llamar a una Función

Para ejecutar una función, simplemente llámás su nombre seguido de paréntesis ().

```
def nombre_de_la_funcion():
    #Cuerpo de la función
    print("Hola, soy una función")

nombre_de_la_funcion() # Output: Hola, soy una función
```

Funciones con Valores de Retorno

Las funciones pueden devolver valores usando la palabra clave `return`. Esto te permite capturar el resultado de la función y usarlo en otra parte de tu código.

```
def suma(a, b):  
    return a + b  
resultado = suma(3, 4)  
print(resultado) # Output: 7
```

Funciones con Parámetros

Las funciones pueden tomar parámetros, que son variables que pasas a la función para que realice operaciones con ellas.

```
def saludar(nombre):  
    print(f"Hola, {nombre}")  
  
saludar("Juan") # Output: Hola, Juan  
saludar("Ana") # Output: Hola, Ana
```

Funciones con Parámetros por Defecto

Pueden definir valores por defecto para los parámetros. Si no se proporciona un valor al llamar a la función, se usará el valor por defecto.

```
def saludar(nombre="amigo"):  
    print(f"Hola, {nombre}")  
  
saludar() # Output: Hola, amigo  
saludar("Luis") # Output: Hola, Luis
```

Funciones con Parámetros Arbitrarios

A veces uno sabe de antemano cuántos parámetros vas a necesitar. Podés usar `*args` para pasar un número arbitrario de argumentos posicionales y `**kwargs` para pasar un número arbitrario de argumentos nombrados.

Un ejemplo de uso de `*args`

```
def sumar_todos(*args):  
    return sum(args)
```

```
print(sumar_todos(1, 2, 3)) # Output: 6
print(sumar_todos(4, 5, 6, 7, 8)) # Output: 30
```

Un ejemplo de uso de `**kwargs`

```
def mostrar_info(**kwargs):
    for clave, valor in kwargs.items():
        print(f"{clave}: {valor}")

mostrar_info(nombre="Ana", edad=30, ciudad="Madrid")
# Output:
# nombre: Ana
# edad: 30
# ciudad: Madrid
```

Retorno de múltiples valores

Las funciones permiten devolver múltiples valores

```
def obtener_coordenadas():
    x = 10
    y = 20
    return x, y

# Llamar a la función y desempaquetar los valores retornados
coord_x, coord_y = obtener_coordenadas()

print(f"Coordenada X: {coord_x}")
print(f"Coordenada Y: {coord_y}")
```

Como sabe el programa que dejamos de ingresar tuplas?

Por el orden.

Archivos

Un archivo es una colección de datos almacenados en un medio de almacenamiento (como un disco duro). Estos datos pueden ser de distintos tipos: texto, binarios, imágenes, videos, entre otros.

En Python, existen funciones y métodos que nos permiten interactuar con archivos, manipulando su contenido de diversas maneras.

Manejo de archivos

Abrir archivos

Abrimos el archivo usando la función `open()`. Al abrir un archivo, Python crea una conexión entre el programa y el archivo en el disco.

```
archivo = open(nombre_archivo, modo)
```

`nombre_archivo`: es el nombre del archivo que queremos abrir (incluyendo su ruta si no está en el mismo directorio).

`modo`: es una cadena de texto que especifica cómo queremos abrir el archivo. Los modos más comunes son:

Modos

- `'r'` : lectura (modo por defecto).
- `'w'` : escritura, sobrescribiendo el archivo si ya existe o creándolo si no.
- `'a'` : anexar, para agregar datos al final del archivo sin borrar su contenido.
- `'b'` : modo binario, que se usa en conjunto con otros modos (por ejemplo, `'rb'` para lectura en modo binario).

```
archivo = open("mi_archivo.txt", "r") # Abre el archivo en modo lectura
```

Leer archivos

Para leer el contenido de un archivo, existen tres métodos principales:

1. `read()` : Lee todo el contenido del archivo y lo devuelve como una cadena.
2. `readline()` : Lee una línea del archivo.
3. `readlines()` : Lee todas las líneas y devuelve una lista de cadenas, cada una representando una línea del archivo.

```
archivo = open("mi_archivo.txt", "r")
contenido = archivo.read()      # Lee todo el contenido
linea = archivo.readline()     # Lee una sola línea
lineas = archivo.readlines()   # Lee todas las líneas en una lista
archivo.close()                # Cierra el archivo
```

Supongamos que tenemos un archivo `datos.txt` con el siguiente contenido:

```
Python es genial.  
La programación es divertida.
```

Para leer cada línea de este archivo:

```
archivo = open("datos.txt", "r")  
for linea in archivo:  
    print(linea.strip()) # strip() elimina los saltos de línea  
archivo.close()
```

Cerrar archivos

Cuando trabajamos con archivos en Python, es importante cerrarlos al terminar para liberar recursos del sistema. Esto se hace con `close()`.

```
archivo.close()
```

Usando el bloque `with` (mejor que `open()`)

El bloque `with` es la forma recomendada de trabajar con archivos en Python, ya que **asegura que el archivo se cierre automáticamente**, incluso si ocurre un error. Esto se conoce como context manager.

```
with open("mi_archivo.txt", "r") as archivo:  
    contenido = archivo.read()  
    print(contenido)  
# No es necesario llamar a archivo.close()
```

Modos de apertura combinados

Además de los modos básicos, podemos combinarlos con `'b'` para trabajar con archivos binarios.

- `'rb'` : Lectura en modo binario.
- `'wb'` : Escritura en modo binario.
- `'ab'` : Anexar en modo binario.

```
with open("imagen.jpg", "rb") as archivo:  
    contenido = archivo.read()  
    print(type(contenido)) # tipo bytes
```

Escribir en archivos

Al escribir en archivos, tenemos dos opciones principales:

- Modo `'w'` : Escribe sobre el archivo. Si el archivo ya existe, borra el contenido.
- Modo `'a'` : Escribe al final del archivo, sin borrar el contenido existente.

Para escribir en un archivo, se utiliza el método `write()` .

```
archivo = open("mi_archivo.txt", "w") # Abre el archivo en modo escritura
archivo.write("Hola, mundo!\n")      # Escribe una línea
archivo.write("Python es divertido.\n")
archivo.close()
```

Important

En modo `'w'` , si el archivo no existe, Python lo crea automáticamente

Extensiones

Trabajando con archivos JSON

Los archivos JSON son comunes para almacenar datos estructurados. Python ofrece el módulo `json` para leer y escribir en formato JSON.

```
import json

datos = {
    "nombre": "Juan",
    "edad": 25,
    "ciudad": "Madrid"
}

with open("datos.json", "w") as archivo:
    json.dump(datos, archivo) # Escribe datos en formato JSON
```

Para poder leer un archivo `.json` lo haremos de la siguiente manera:

```
import json

with open("datos.json", "r") as archivo:
```

```
datos = json.load(archivo) # Carga el contenido como un diccionario
print(datos)
```

Trabajando con archivos CSV

Los archivos CSV son un formato de texto estructurado que se usa mucho en hojas de cálculo. Python incluye el módulo `csv` para trabajar con archivos CSV.

```
import csv

datos = [
    ["Nombre", "Edad", "Ciudad"],
    ["Juan", 25, "Madrid"],
    ["Ana", 28, "Barcelona"]
]

with open("datos.csv", "w", newline="") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerows(datos) # Escribe múltiples filas
```

Para leer los archivos csv podemos hacerlo de la siguiente manera:

```
import csv

with open("datos.csv", "r") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)
```

Archivos y directorios

Para trabajar con archivos y directorios, Python incluye el módulo `os` y `os.path`.

Obtener la ruta del directorio actual

```
import os
ruta_actual = os.getcwd()
print(ruta_actual)
```

Listar archivos en un directorio

```
import os
archivos = os.listdir(".")
print(archivos)
```

Verificar si un archivo existe

```
import os
if os.path.exists("mi_archivo.txt"):
    print("El archivo existe.")
```

Excepciones

Una excepción es un evento que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de las instrucciones. Python tiene varios tipos de excepciones ya integradas, como:

- `ValueError` : ocurre cuando se recibe un valor de tipo incorrecto
- `TypeError` : ocurre cuando se usa un tipo de dato inapropiado
- `FileNotFoundError` : ocurre cuando se intenta abrir un archivo que no existe

Cuando surge una excepción, el programa se detiene. Sin embargo, podemos "manejar" esa excepción para evitar que el programa falle.

Sintaxis Básica `try` y `except`

Para capturar y manejar errores en Python, usamos la estructura `try-except` . Básicamente, le decimos a Python: "Intenta hacer algo, y si algo sale mal, haz otra cosa".

```
try:
    numero = int(input("Ingresa un número: "))
    print(f"El número ingresado es {numero}")
except ValueError:
    print("Eso no es un número válido.")
```

`try` contiene el código que puede fallar.

`except` contiene el código que se ejecuta si ocurre el error. En este caso, si el usuario ingresa un valor que no se puede convertir a entero, `ValueError` se "atrapa" y se ejecuta el mensaje de error.

Capturar Múltiples Excepciones

A veces, diferentes tipos de errores pueden surgir en un mismo bloque de código. Podemos usar varios `except` para cada tipo de error, lo cual nos permite manejar cada caso de forma específica.

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
    numero = int(contenido)
    print(numero)
except FileNotFoundError:
    print("El archivo no existe.")
except ValueError:
    print("El contenido del archivo no es un número.")
```

Si el archivo `datos.txt` no existe, se captura `FileNotFoundError`.

Si el contenido no es un número, se captura `ValueError`.

Else y Finally

`else` : se usa para definir código que se ejecutará solo si no se produce ninguna excepción en el bloque `try`.

`finally` : se ejecuta siempre, independientemente de si ocurre o no una excepción. Es útil para tareas de limpieza, como cerrar archivos o liberar recursos.

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
except FileNotFoundError:
    print("No se pudo abrir el archivo.")
else:
    print("Archivo leído correctamente.")
    print(contenido)
finally:
    print("Ejecución finalizada.")
```

Si `datos.txt` existe, se lee y se imprime su contenido.

El mensaje “Ejecución finalizada” se muestra siempre, ocurra o no un error.

Lanzar Excepciones `raise`

La instrucción `raise` permite lanzar una excepción manualmente. Esto es útil cuando queremos validar condiciones específicas en nuestro código.

```
def dividir(a, b):
    if b == 0:
        raise ZeroDivisionError("No se puede dividir entre cero.")
    return a / b

try:
    resultado = dividir(10, 0)
    print(resultado)
except ZeroDivisionError as e:
    print(e)
```

Excepciones Personalizadas

Python permite crear nuestras propias excepciones para situaciones específicas en nuestro programa. Esto es útil si queremos distinguir entre errores propios del negocio o dominio del programa y los errores generales del sistema.

```
class EdadInvalidaError(Exception):
    """Excepción lanzada cuando se ingresa una edad inválida"""
    pass

def verificar_edad(edad):
    if edad < 0:
        raise EdadInvalidaError("La edad no puede ser negativa.")
    print("Edad válida.")

try:
    verificar_edad(-5)
except EdadInvalidaError as e:
    print(e)
```

Ejemplo Completo

Este ejemplo es un flujo completo de manejo de errores en Python:

- Intenta abrir y procesar el archivo.
- Captura errores específicos para `FileNotFoundError` y `ValueError`.
- Cierra el archivo en el bloque `finally` para asegurarse de que el recurso se libere.

```
def procesar_archivo(ruta):
    try:
        archivo = open(ruta, "r")
        contenido = archivo.read()
        numero = int(contenido.strip())
    except FileNotFoundError:
        print("Error: No se encontró el archivo.")
    except ValueError:
        print("Error: El contenido del archivo no es un número válido.")
    else:
        print(f"El número en el archivo es {numero}")
    finally:
        archivo.close()
        print("Archivo cerrado.")

procesar_archivo("datos.txt")
```

Numpy

<https://numpy.org/devdocs/reference/generated/numpy.std.html>

Motivación

Las listas en Python son flexibles, pero lentas para operaciones matemáticas grandes. Necesitamos trabajar con grandes volúmenes de datos de forma rápida.

Usando listas

```
lista = [i for i in range(1000000)]
suma = [x+5 for x in lista]
```

Usando NumPy

```
import numpy as np
arr = np.arange(1000000)
suma = arr + 5
```

Introducción

NumPy (Numerical Python) es una biblioteca de Python utilizada para realizar operaciones matemáticas y estadísticas con grandes conjuntos de datos.

- Librería fundamental para computación científica.
- Maneja estructuras llamadas `ndarrays` (n-dimensional arrays).
- Soporta operaciones vectorizadas (más eficientes que bucles).

```
import numpy as np
a = np.array([1, 2, 3])
print(a * 2) # [2 4 6]
```

Creación de Arrays

Muy útiles para inicializar datos.

```
np.array([1, 2, 3])
np.zeros((2, 3)) # Matriz 2x3 llena de ceros
np.ones((3, 3)) # Matriz 3x3 de unos
np.arange(0, 10, 2) # range [0, 2, 4, 6, 8]
np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1.] (inicio, fin, cantidad de
numeros) genera una cantidad de numeros especificada, inicio y fin inclusive
```

Propiedades de los Arrays

A diferencia de las listas, los arrays son homogéneos (todos los elementos del mismo tipo).

```
arr = np.array([[1,2,3],[4,5,6]])
print(arr.shape) # (2,3)
print(arr.ndim) # 2 dimensiones
print(arr.dtype) # tipo de dato
```

Indexado y Slicing

Funciona parecido a las listas, pero con más potencia en varias dimensiones.

```
arr = np.array([10,20,30,40,50])
print(arr[1]) # 20
print(arr[1:4]) # [20 30 40]
print(arr[:,2]) # [10 30 50]
```

Operaciones Matemáticas

- Operaciones elemento a elemento: + - * /
- Broadcasting: NumPy ajusta automáticamente dimensiones

```
a = np.array([1,2,3])
b = np.array([10])
print(a + b) # [11 12 13]
```

Funciones NumPy útiles

Estadísticas:

```
arr = np.random.randint(1, 100, 10)
print(np.mean(arr), np.std(arr), np.min(arr), np.max(arr))
```

Aleatorios:

```
np.random.normal(0, 1, 1000) # distribución normal
np.random.rand(3, 3) # números aleatorios en matriz 3x3 de 0 a 1
```

Matplotlib

Es una biblioteca de Python para crear gráficos estáticos, animados e interactivos. Fue creada por John D. Hunter y es ampliamente utilizada en la comunidad científica y de análisis de datos debido a su flexibilidad y potencia.

Características Principales

- Librería para visualización científica
- Permite crear gráficos 2D y 3D

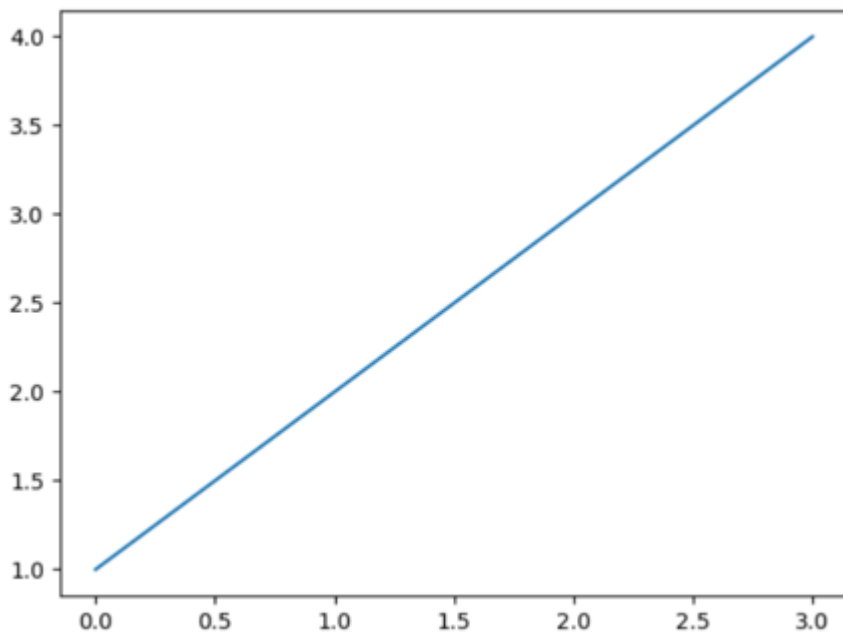
Importación:

```
import matplotlib.pyplot as plt
```

Primer gráfico

Muestra una curva simple con valores en el eje Y.

```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4])
plt.show()
```



Creación de un Gráfico Simple

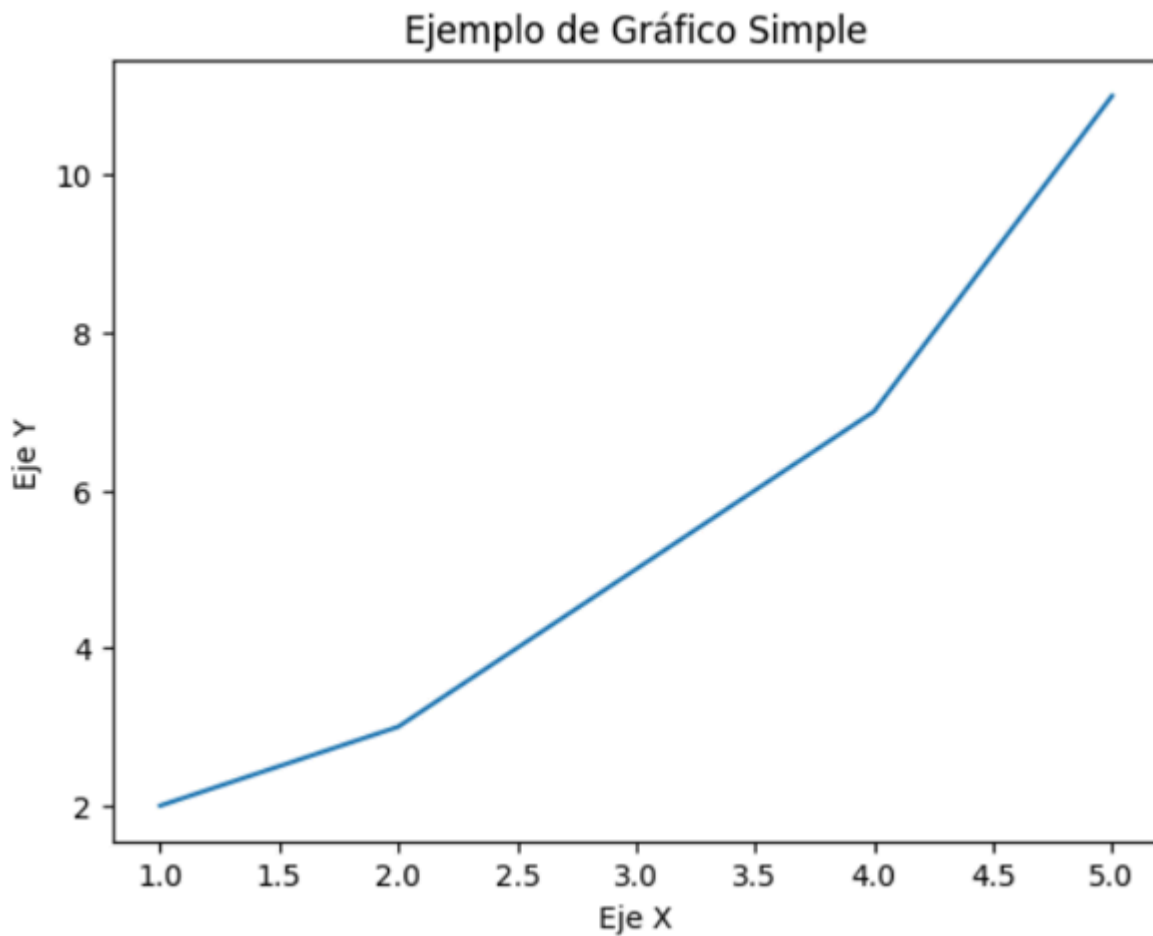
```
import matplotlib.pyplot as plt

# Datos
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# Crear el gráfico
plt.plot(x, y)

# Añadir título y etiquetas
plt.title("Ejemplo de Gráfico Simple")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")

# Mostrar el gráfico
plt.show()
```



Creación de Gráfico de Barras

```
import matplotlib.pyplot as plt

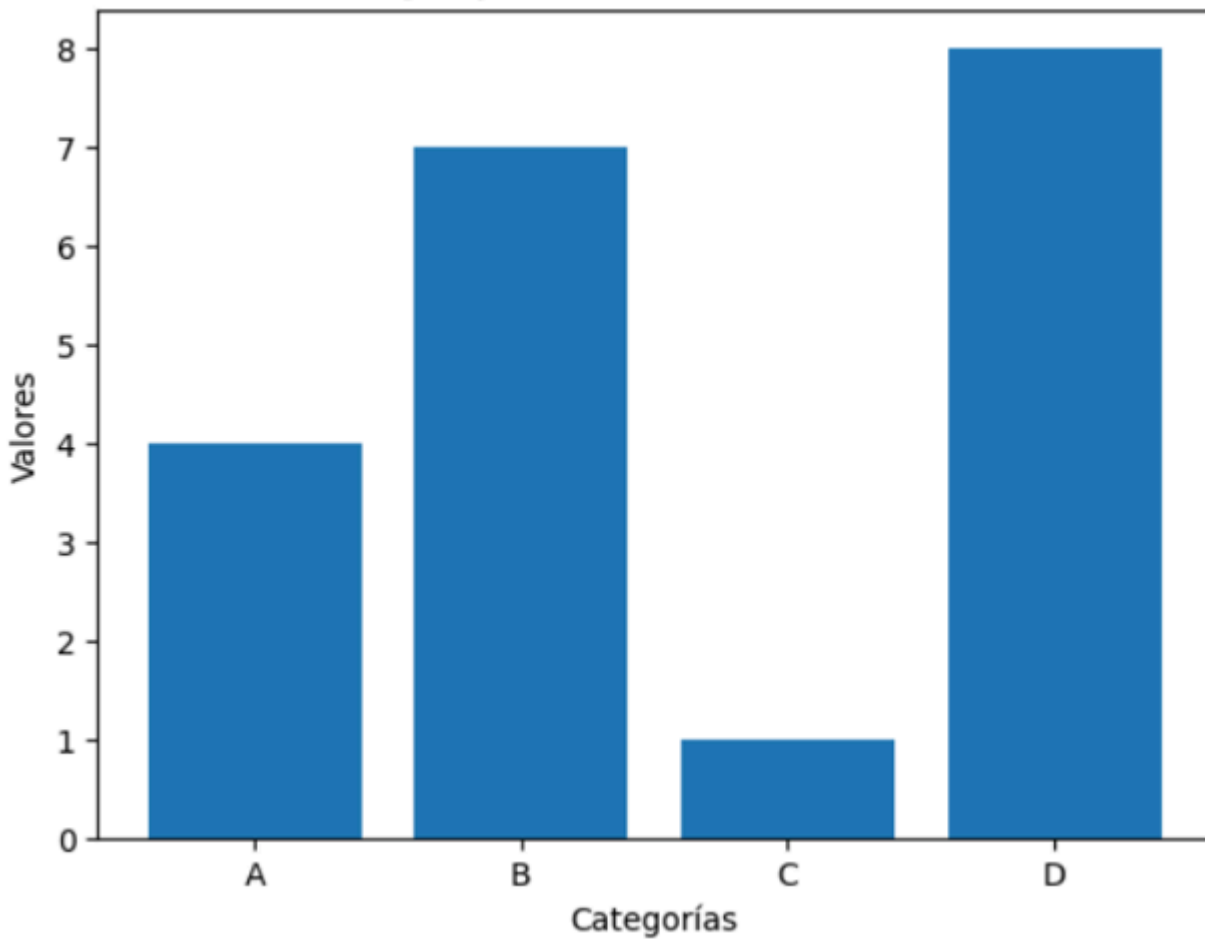
# Datos
categories = ['A', 'B', 'C', 'D']
values = [4, 7, 1, 8]

# Crear el gráfico de barras
plt.bar(categories, values)

# Añadir título y etiquetas
plt.title("Ejemplo de Gráfico de Barras")
plt.xlabel("Categorías")
plt.ylabel("Valores")

# Mostrar el gráfico
plt.show()
```

Ejemplo de Gráfico de Barras



Creación de Gráfico de Histograma

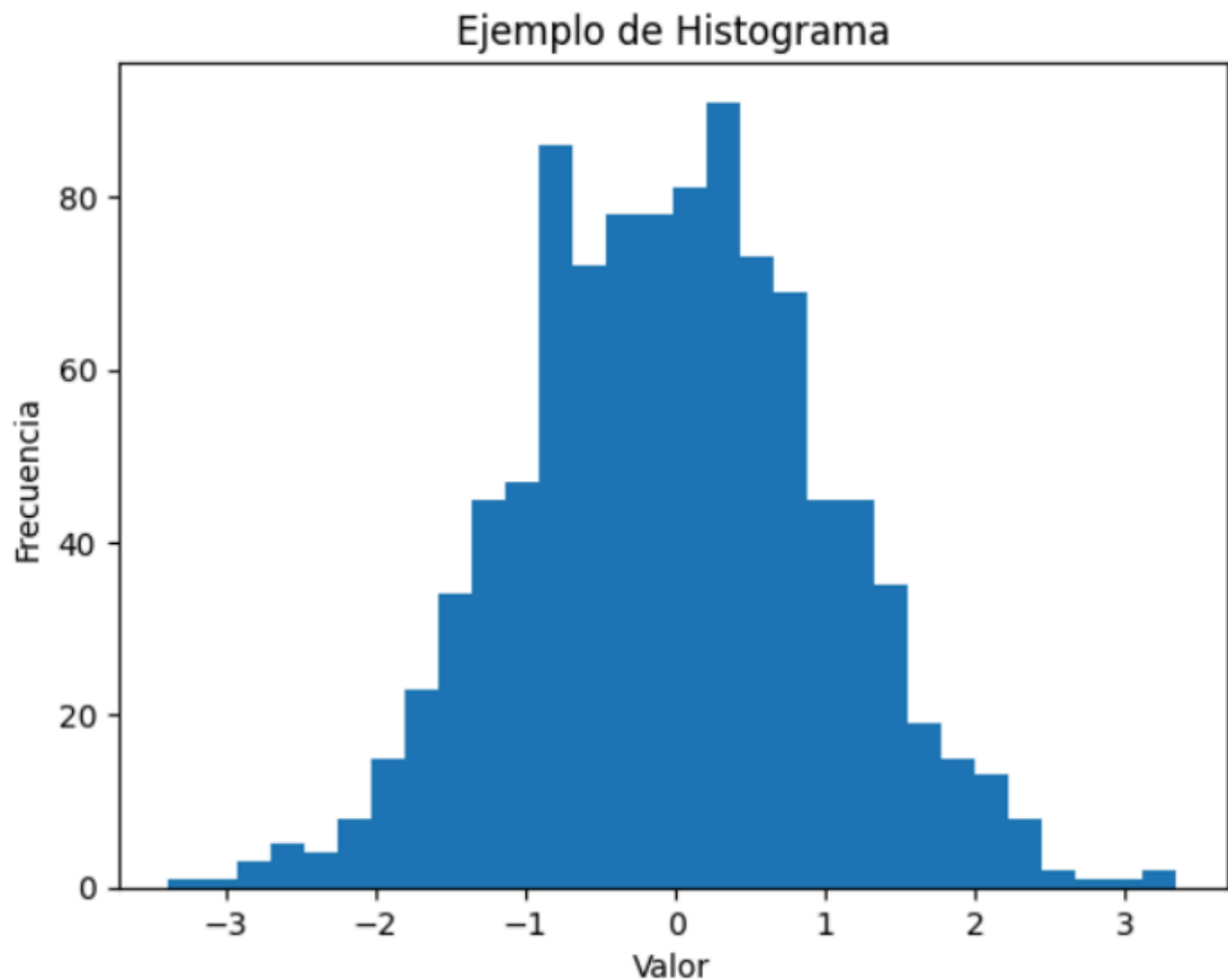
```
import numpy as np

# Datos
data = np.random.randn(1000)

# Crear el histograma
plt.hist(data, bins=30)

# Añadir título y etiquetas
plt.title("Ejemplo de Histograma")
plt.xlabel("Valor")
plt.ylabel("Frecuencia")

# Mostrar el gráfico
plt.show()
```



Creación de Gráfico de Dispersión

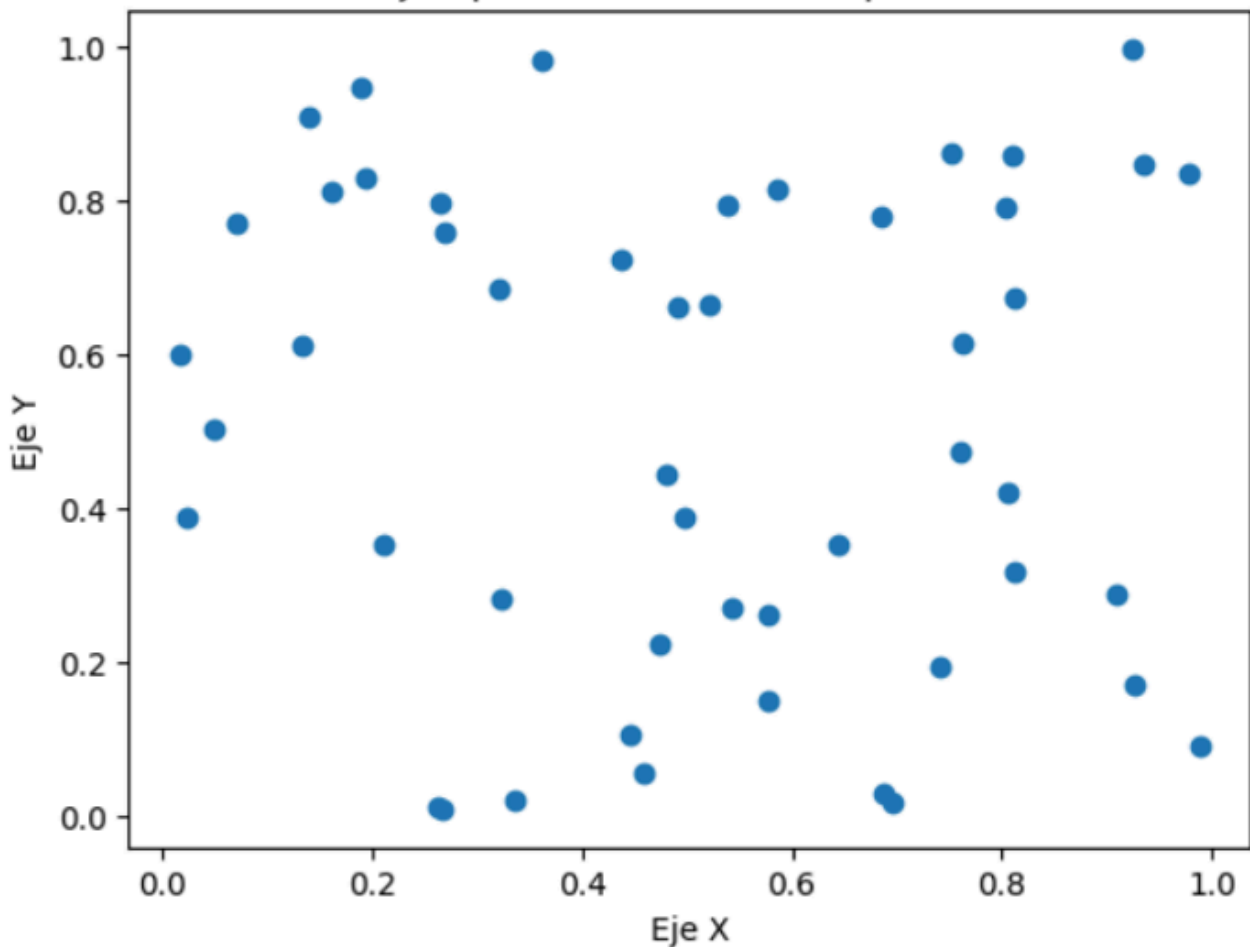
```
# Datos
x = np.random.rand(50)
y = np.random.rand(50)

# Crear el gráfico de dispersión
plt.scatter(x, y)

# Añadir título y etiquetas
plt.title("Ejemplo de Gráfico de Dispersión")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")

# Mostrar el gráfico
plt.show()
```

Ejemplo de Gráfico de Dispersión



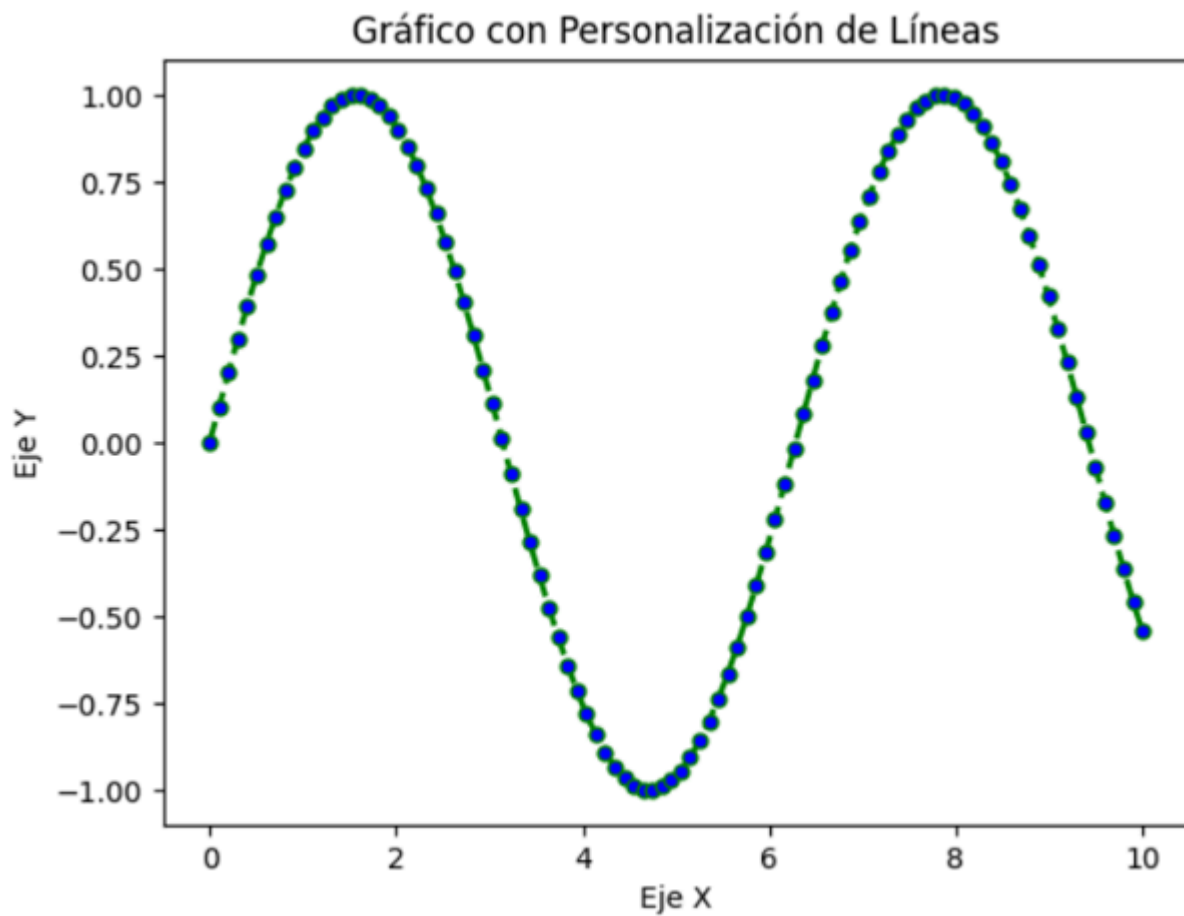
Personalización Avanzada

```
# Datos
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Crear el gráfico con personalización
plt.plot(x, y, color='green', linestyle='--', linewidth=2, marker='o',
markerfacecolor='blue', markersize=5)

# Añadir título y etiquetas
plt.title("Gráfico con Personalización de Líneas")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")

# Mostrar el gráfico
plt.show()
```



Subplots

```
# Datos
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Crear los subplots
fig, (ax1, ax2) = plt.subplots(2, 1)

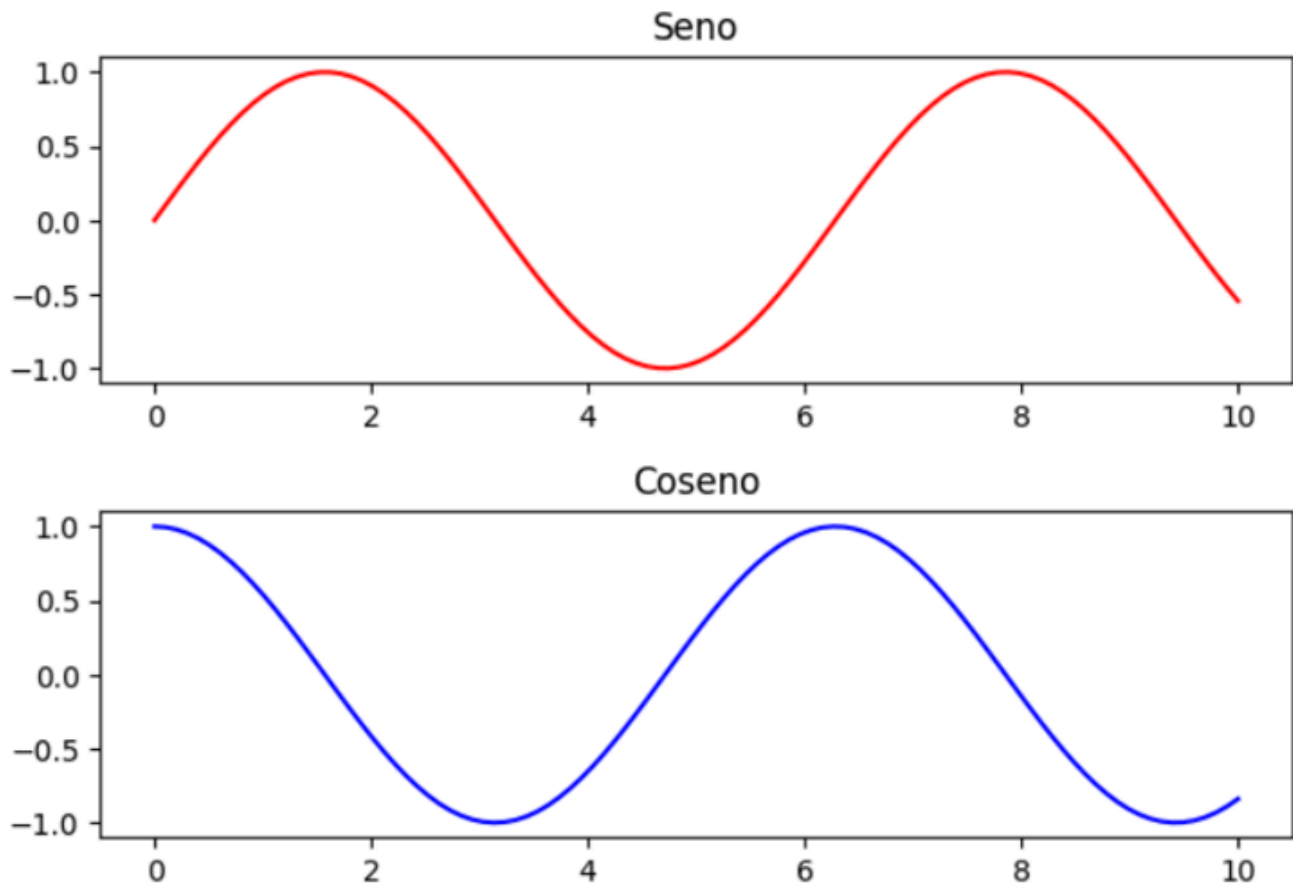
# Primer subplot
ax1.plot(x, y1, 'r')
ax1.set_title('Seno')

# Segundo subplot
ax2.plot(x, y2, 'b')
ax2.set_title('Coseno')

# Añadir título general y ajustar el layout
fig.suptitle('Ejemplo de Subplots')
plt.tight_layout()
```

```
# Mostrar el gráfico  
plt.show()
```

Ejemplo de Subplots



-
1. En OR acumulado, el símbolo es $\geq$, pero es imposible ponerlo correctamente por la forma que esta formateada la tabla ↩